

Universidad Mayor de San Andrés

Facultad de Ciencias Puras y Naturales
Carrera de Informática



Tráfico Inteligente

Análisis no supervisado de la congestión vial
en La Paz y El Alto, Bolivia
mediante Aprendizaje Automático No Supervisado

Maximiliano Gómez Mallo
14480221

Materia: Machine Learning
Docente: Menfy Morales Rios
Fecha: La Paz, Bolivia — Mayo 2026

Índice

1	Introducción	3
1.1	¿Por qué aprendizaje no supervisado?	3
2	Adquisición de datos	3
2.1	Medición de congestión: el jam factor	3
2.2	Red vial: OSMnx	4
2.3	Días y horas recolectados	6
3	Construcción de la feature matrix	6
3.1	Concepto: la tabla de datos	6
4	Modelos no supervisados	7
4.1	PCA — Reducción de dimensionalidad	7
4.1.1	Intuición: la sombra más informativa	7
4.1.2	Paso a paso	8
4.1.3	Matemática	8
4.2	UMAP — Visualización en 2D	9
4.2.1	Intuición: el mapa del terreno	9
4.2.2	El parámetro n_neighbors: local vs. global	9
4.3	DBSCAN — Clustering espacial geográfico	10
4.3.1	Intuición: islas de densidad	10
4.3.2	Core points, border points y ruido	11
4.3.3	¿Por qué la distancia haversine?	11
4.4	K-Means — Clustering temporal	11
4.4.1	Intuición: el proceso de asignación iterativa	11
4.4.2	La función de costo	12
4.4.3	Elección de k: el método del codo	12
4.4.4	Resultados del clustering	13
4.5	NMF — Síntesis de días faltantes	14
4.5.1	El problema	14
4.5.2	Intuición: los arquetipos del tráfico	14
4.5.3	Matemática de NMF	14
4.5.4	Pipeline NMF + PCHIP	15
4.6	PCHIP — Interpolación día-de-semana	15
4.6.1	Intuición: la curva que no oscila	15
4.6.2	Validación cruzada interna	16
4.6.3	Nivel de confianza por día	17

5	Pipeline completo y decisiones de diseño	17
5.1	Flujo de datos	17
5.2	Decisiones de diseño no obvias	17
6	Dashboard interactivo	18
6.1	Arquitectura	18
6.2	Extensión de calles por nombre	18
6.3	Modos de visualización	19
6.4	Transparencia: datos estimados	19
7	Resultados	19
7.1	Patrones encontrados	19
7.2	Limitaciones	20
8	Conclusiones	20
A	Dependencias del proyecto	20
B	Estructura de archivos	21

1. Introducción

La ciudad de La Paz y su vecina El Alto conforman una de las conurbaciones más complejas de América del Sur desde el punto de vista vial. La Paz descansa en una hoya montañosa a unos 3 600 m.s.n.m., mientras que El Alto se extiende en el altiplano a 4 100 m.s.n.m.; ambas ciudades están conectadas por apenas unas pocas autopistas de alta pendiente, lo que convierte cualquier incidente en una cascada de congestión que puede paralizar barrios enteros durante horas.

Objetivo del proyecto. Clasificar automáticamente segmentos de carretera según su patrón de congestión *sin utilizar etiquetas previas* —es decir, sin que nadie haya definido de antemano qué significa “alta congestión” o “zona tranquila”. Para ello aplicamos un conjunto de algoritmos de **aprendizaje no supervisado**: reducción de dimensionalidad (PCA, UMAP), clustering espacial (DBSCAN) y temporal (K-Means), y síntesis de datos faltantes (NMF + PCHIP).

El resultado final es un **dashboard interactivo** que permite a cualquier usuario explorar el comportamiento del tráfico hora a hora, día a día, y comparar ambas ciudades de forma visual e intuitiva.

1.1. ¿Por qué aprendizaje no supervisado?

En el aprendizaje supervisado, alguien debe etiquetar los datos previamente: “esta calle es alta congestión”, “esta otra es tranquila”. Esto requiere expertos, tiempo y dinero.

En el aprendizaje **no supervisado**, el algoritmo descubre la estructura por sí mismo, preguntando: *¿qué segmentos se parecen entre sí?* Esta pregunta es mucho más poderosa cuando los datos son complejos: en lugar de clasificar calles en dos categorías rígidas, el modelo puede descubrir grupos más sutiles que un humano nunca habría definido a priori.

2. Adquisición de datos

2.1. Medición de congestión: el jam factor

Para medir la congestión usamos la *Distance Matrix API* de Google, que devuelve el tiempo de viaje entre pares de puntos con y sin tráfico. De estos dos valores calculamos el **jam factor**:

$$\text{jam_factor} = \frac{\text{duración con tráfico}}{\text{duración típica}} \quad (1)$$

¿Cómo interpretar el jam factor?

Un jam_factor de 1,0 significa tráfico fluido: el viaje tarda lo esperado. Un valor de 2,5 indica que el trayecto toma *2,5 veces* más de lo normal. En La Paz, los valores oscilan entre 1,0 (madrugada, calles vacías) y $\approx 3,5$ (hora pico, centros comerciales).

2.2. Red vial: OSMnx

Para definir *qué* calles consultar, usamos **OSMnx**, una librería Python que descarga el grafo vial de OpenStreetMap gratuitamente. Cada segmento (arista del grafo) tiene coordenadas GPS de inicio y fin, longitud en metros y tipo de vía.

El grafo de La Paz contiene $\approx 37\,100$ segmentos de calle. Consultar todos excedería el presupuesto de la API, así que **muestreamos 250 segmentos por zona** estratificando por tipo de vía para garantizar que autopistas, avenidas y calles residenciales estén representadas proporcionalmente.

Red vial vehicular — La Paz, Bolivia

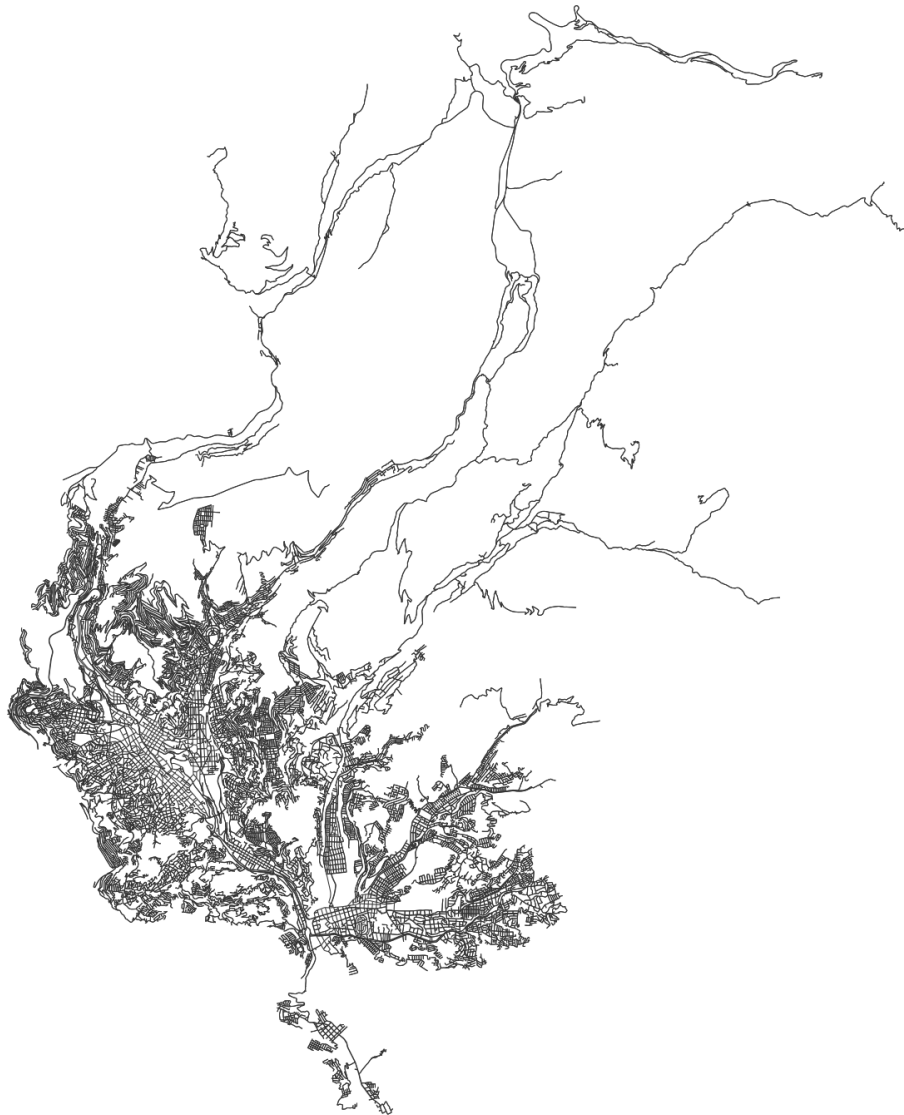


Figura 1: Red vial completa de La Paz descargada con OSMnx (37 100 segmentos). El grafo captura desde autopistas hasta calles locales.

```
1 import osmnx as ox
2
3 G = ox.graph_from_place("La Paz, Bolivia", network_type="drive")
4 edges = ox.graph_to_gdfs(G, nodes=False) # 37 100 aristas
5
6 # Muestreo estratificado por tipo de via
7 sample = (
8     edges.groupby("highway", group_keys=False)
9     .apply(lambda g: g.sample(min(len(g), 30), random_state=42))
10 )
11 sample = sample.sample(250, random_state=42).reset_index()
```

Listing 1: Descarga del grafo vial y muestreo estratificado.

2.3. Días y horas recolectados

Por restricciones de cuota de la API se recolectaron **4 días** representativos de la semana: lunes (día laboral típico), miércoles (mitad de semana), viernes (día laboral con tráfico extra) y sábado (fin de semana). Para cada día se consultaron las 24 horas, obteniendo un retrato completo del comportamiento diario.

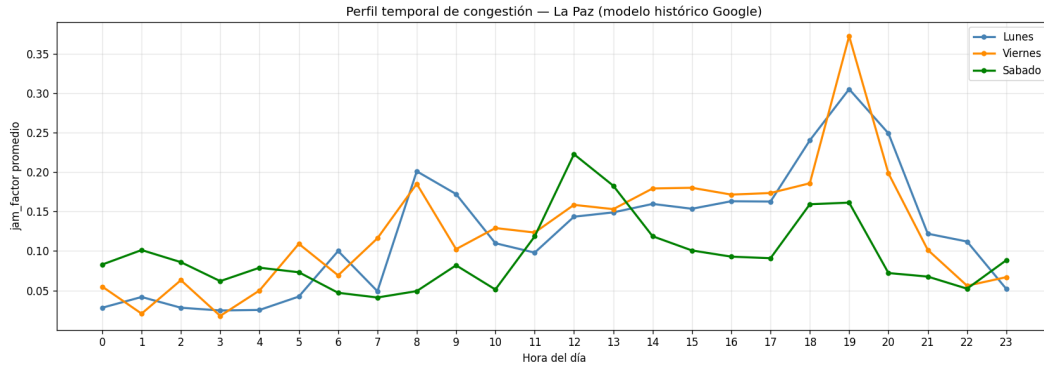


Figura 2: Perfiles de congestión por hora del día para los distintos días recolectados. Se aprecia el pico matutino (7–9 h) y vespertino (17–19 h) característicos de La Paz, y el patrón más suave del sábado.

3. Construcción de la feature matrix

3.1. Concepto: la tabla de datos

Antes de aplicar cualquier modelo, necesitamos representar cada segmento vial como un **vector numérico**: una lista de números que describa su comportamiento de tráfico. A esta tabla de vectores la llamamos **feature matrix** (matriz de características).

Cada fila es un segmento; cada columna es el `jam_factor` promedio en una combinación {día, hora}:

$$X \in \mathbb{R}^{250 \times 96} \quad \text{donde} \quad X_{i,d,24+h} = \text{jam_factor del segmento } i \text{ el día } d \text{ a las } h \text{ horas} \quad (2)$$

Los 96 columnas provienen de 4 días $\times$ 24 h/día.

¿Por qué usar las 24 horas y no solo la hora pico? Las horas pico (7–9 h, 17–19 h) capturan el máximo de congestión, pero el *patrón completo* del día —cómo sube y baja el tráfico a lo largo de las 24 horas— distingue mejor los tipos de calle. Una avenida comercial está congestionada también al mediodía; una autopista de salida solo en horas pico. Al incluir todas las horas, los modelos pueden capturar estas diferencias sutiles.

```
1 DIAS = ["lunes", "miercoles", "viernes", "sabado"]
2 mat = segs.set_index("segment_id").copy()
3
```

```

4 for dia in DIAS:
5     for h in range(24):
6         f = traffic_dir / f"traffic_{dia}_{h:02d}.csv"
7         col = f"jam_{dia}_{h:02d}"
8         df = pd.read_csv(f)[["segment_id", "jam_factor"]]
9         mat[col] = df.set_index("segment_id")["jam_factor"]
10
11 mat = mat.fillna(0.0)    # segmentos sin dato -> fluido (1.0
                           # normalizado)
12 mat.to_csv(output_csv)

```

Listing 2: Construcción de la feature matrix (script 03).

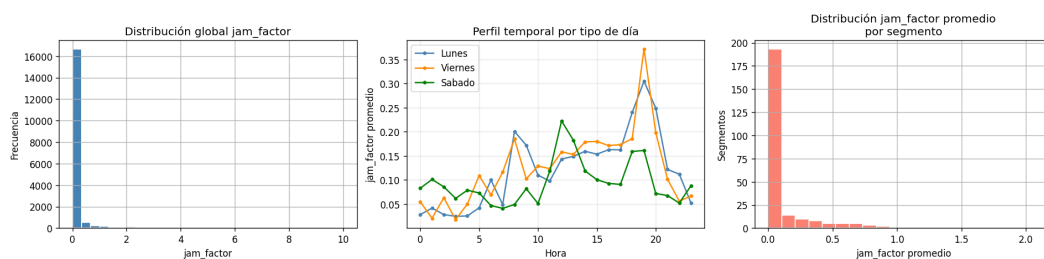


Figura 3: Distribución global del jam_factor y perfiles diarios promedio. La mayoría de los valores se concentran cerca de 1,0 (tráfico fluido), con una cola larga hacia valores altos durante las horas pico.

4. Modelos no supervisados

El aprendizaje no supervisado parte de una premisa sencilla: *los datos tienen estructura interna* que podemos descubrir sin necesidad de que alguien nos diga cuál es la respuesta correcta. En este proyecto usamos seis técnicas complementarias, cada una respondiendo una pregunta diferente sobre los datos.

4.1. PCA — Reducción de dimensionalidad

4.1.1. Intuición: la sombra más informativa

Imaginemos que sostenemos un objeto tridimensional (una figura irregular) frente a una fuente de luz y proyectamos su sombra en una pared. Según desde qué ángulo pongamos la luz, la sombra mostrará más o menos detalles del objeto. **PCA** (*Principal Component Analysis*) busca *el ángulo de iluminación que produce la sombra más informativa*: la proyección que preserve la mayor cantidad de variación entre los puntos.

En nuestro caso, los 250 segmentos son objetos en un espacio de 96 dimensiones (una por hora-día). Visualizar 96 dimensiones es imposible, y muchas de ellas contienen información redundante: la congestión del lunes a las 8 h y del miércoles a las 8 h están

altamente correlacionadas. PCA encuentra las pocas “direcciones independientes” que capturan la mayor parte de esa variación.

4.1.2. Paso a paso

1. **Estandarizar:** cada columna se centra en 0 y se escala a varianza 1, para que no dominen las horas con valores más altos.
2. **Calcular la covarianza:** se mide cuánto varían juntas cada par de columnas. El resultado es la *matriz de covarianza* $\Sigma \in \mathbb{R}^{p \times p}$.
3. **Descomponer:** se calculan los *eigenvectores* de Σ . Cada eigenvector v_k es una dirección en el espacio original, y su eigenvalor λ_k indica cuánta varianza captura.
4. **Proyectar:** cada segmento se representa como sus coordenadas en las K primeras direcciones $\rightarrow$ de 96 a 13 números.

4.1.3. Matemática

Dado $X \in \mathbb{R}^{n \times p}$ (estandarizado), la varianza explicada por las primeras K componentes es:

$$VE(K) = \frac{\sum_{k=1}^K \lambda_k}{\sum_{k=1}^p \lambda_k}, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0 \quad (3)$$

Con $K = 13$ se captura entre el 86% (La Paz) y el 96% (El Alto) de la varianza total. Esto significa que 13 números por segmento describen casi tan bien su comportamiento como los 96 originales.

¿Qué captura PC1? La primera componente principal correlaciona fuertemente con el nivel general de congestión: segmentos que son congestionados en todas las horas y días tienen un PC1 alto. PC2 captura la diferencia entre el patrón de días laborales y el sábado. Las siguientes componentes capturan patrones cada vez más sutiles (diferencias entre horas, entre zonas, etc.).

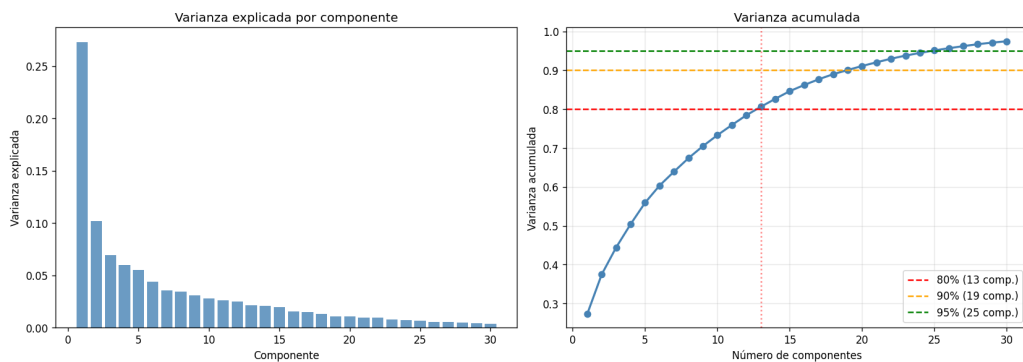


Figura 4: Izquierda: varianza explicada por cada componente principal (gráfico de scree). Derecha: varianza acumulada. Con 13 componentes se supera el umbral del 85%.

```
1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3
4 X_std = StandardScaler().fit_transform(feat.values)
5 pca = PCA(n_components=13, random_state=42)
6 X_pca = pca.fit_transform(X_std) # (250, 13)
7 var_ac = pca.explained_variance_ratio_.cumsum()[-1]
8 print(f"Varianza acumulada: {var_ac:.1%}")
9 # La Paz: 86.8%
10 # El Alto: 96.1%
```

Listing 3: PCA con sklearn.

4.2. UMAP — Visualización en 2D

4.2.1. Intuición: el mapa del terreno

PCA es como tomar una fotografía aérea de la nube de puntos: proyecta de forma *lineal*, sin doblar el espacio. Pero los datos reales muchas veces viven en una *variedad* (manifold): una superficie curva embebida en alta dimensión, como la superficie de la Tierra dentro del espacio 3D.

UMAP (*Uniform Manifold Approximation and Projection*) es como hacer un *mapa cartográfico* de esa superficie curva: aplanar la variedad en 2D intentando que los puntos que eran vecinos en alta dimensión sigan siendo vecinos en el plano.

La diferencia clave con t-SNE (otro método popular) es que UMAP **preserva mejor la estructura global**: no solo agrupa localmente, sino que mantiene la distancia relativa entre grupos. Además es considerablemente más rápido.

4.2.2. El parámetro *n_neighbors*: local vs. global

Imagine que en un barrio nuevo solo conoce a sus 3 vecinos más cercanos. Su idea del barrio será muy local —solo ve lo que pasa en su manzana. Si en cambio conoce a sus 50 vecinos más cercanos, tiene una visión más global —puede detectar patrones de toda la cuadra.

El parámetro `n_neighbors` en UMAP funciona igual: un valor pequeño (ej. 5) hace que UMAP se concentre en estructura muy fina y local; uno grande (ej. 50) captura la estructura global. Con `n_neighbors = 15` para 250 puntos obtenemos un equilibrio adecuado.

Parámetro	Valor	Efecto
n_neighbors	15	equilibrio local/global para 250 puntos
min_dist	0.1	separación mínima, evita solapamiento
n_components	2	proyección en 2 dimensiones

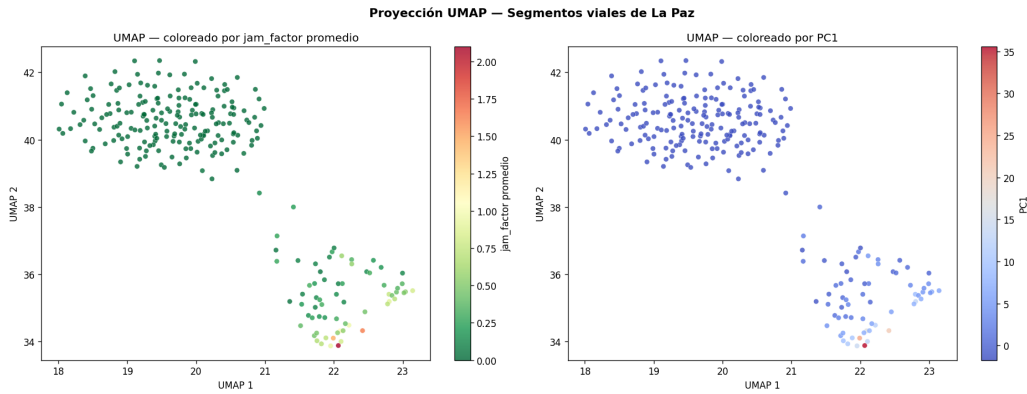


Figura 5: Proyección UMAP de los 250 segmentos de La Paz. Izquierda: coloreado por jam_factor medio (rojo = más congestionado). Derecha: coloreado por PC1. Los dos grupos visibles corresponden a los clusters de alta y baja congestión descubiertos por K-Means.

```

1 import umap
2
3 reducer = umap.UMAP(n_neighbors=15, min_dist=0.1,
4                     n_components=2, random_state=42)
5 X_umap = reducer.fit_transform(X_pca)      # (250, 2)

```

Listing 4: UMAP sobre las componentes PCA.

4.3. DBSCAN — Clustering espacial geográfico

4.3.1. Intuición: islas de densidad

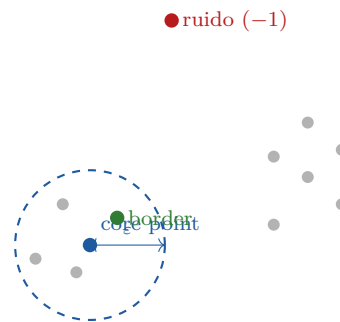
DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) parte de una idea geográfica muy natural: un **barrio** es una zona donde las casas están relativamente juntas, separada de otros barrios por áreas más despobladas.

El algoritmo busca “islas” de puntos densos separadas por zonas vacías. Las dos ventajas clave sobre K-Means son:

1. **No requiere especificar k:** el algoritmo encuentra cuántos clusters hay automáticamente.
2. **Detecta outliers:** los puntos aislados que no pertenecen a ningún grupo se marcan como ruido (valor -1).

En este proyecto lo aplicamos sobre las *coordenadas GPS* de los segmentos para identificar zonas geográficas homogéneas de la ciudad.

4.3.2. Core points, border points y ruido



Un punto es **core point** si tiene al menos `min_samples` vecinos en un radio ε . Los puntos en la frontera de un core point (pero que no lo son ellos mismos) son **border points**. Los puntos que no tienen suficientes vecinos son **ruido**.

4.3.3. ¿Por qué la distancia haversine?

La distancia euclídea entre coordenadas GPS (Δlat , Δlon) es imprecisa: 1 grado de longitud en el ecuador mide ~ 111 km, pero en latitud -16° (La Paz) mide ~ 107 km. La **fórmula haversine** calcula la distancia real sobre la esfera terrestre:

$$d = 2R \arcsin\left(\sqrt{\sin^2\frac{\Delta\phi}{2} + \cos\phi_1 \cos\phi_2 \sin^2\frac{\Delta\lambda}{2}}\right) \quad (4)$$

donde $R = 6\,371\,000$ m, ϕ es latitud y λ longitud en radianes.

```

1 from sklearn.cluster import DBSCAN
2 import numpy as np
3
4 EARTH_RADIUS_M = 6_371_000
5 coords_rad = np.radians(segs[["lat", "lon"]].values)
6
7 db = DBSCAN(
8     eps=500 / EARTH_RADIUS_M,    # 500 m en radianes
9     min_samples=5,
10    metric="haversine"
11 )
12 cluster_espacial = db.fit_predict(coords_rad)

```

Listing 5: DBSCAN haversine con sklearn.

4.4. K-Means — Clustering temporal

4.4.1. Intuición: el proceso de asignación iterativa

Imagine que tiene que dividir 250 alumnos en 2 grupos equilibrados según su rendimiento en 96 exámenes. Una estrategia es:

1. Elegir 2 alumnos al azar como “representantes” iniciales.
2. Asignar cada alumno al representante más parecido.
3. Reemplazar cada representante por el promedio de su grupo.
4. Repetir desde el paso 2 hasta que los grupos no cambien.

K-Means hace exactamente esto, pero con vectores de 96 dimensiones (perfil completo de tráfico por hora-día) y como medida de similitud usa la distancia euclídea.

4.4.2. La función de costo

K-Means minimiza la suma total de distancias al cuadrado entre cada punto y el centroide de su cluster:

$$J = \sum_{j=1}^k \sum_{i \in C_j} \|x_i - \mu_j\|^2 \quad (5)$$

donde μ_j es el centroide del cluster j y C_j el conjunto de puntos asignados a ese cluster.

4.4.3. Elección de k : el método del codo

Para elegir k graficamos la función de costo J en función del número de clusters. Al agregar más clusters, J siempre baja (más grupos $\rightarrow$ segmentos más cerca de su centroide). Pero hay un punto donde la mejora se vuelve marginal: el “codo” de la curva. Complementamos con el **índice de silueta**, que mide qué tan bien separados están los clusters (1 = perfectamente separados, 0 = solapados).

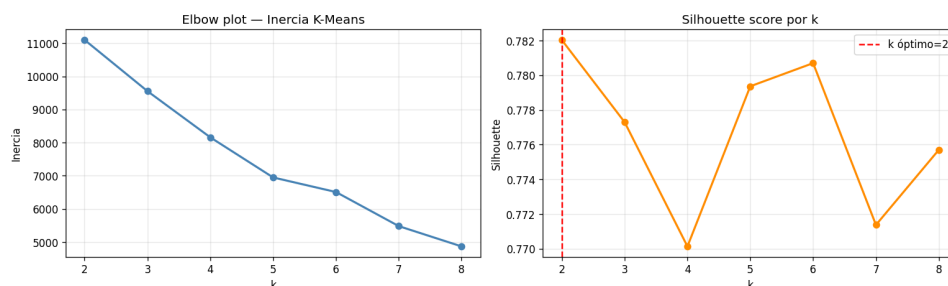


Figura 6: Método del codo (inercia vs. k) e índice de silueta. Ambos indicadores señalan $k = 2$ como el número óptimo de clusters: el codo se forma en $k = 2$ y la silueta alcanza su máximo.

```

1 from sklearn.cluster import KMeans
2 from sklearn.metrics import silhouette_score
3
4 # Elegir k evaluando inercia y silueta
5 for k in range(2, 8):
6     km = KMeans(n_clusters=k, random_state=42, n_init=20)

```

```

7     lbl = km.fit_predict(X_std)
8     sil = silhouette_score(X_std, lbl)
9     print(f"k={k}: inercia={km.inertia_:.1f}, silueta={sil:.3f}")
10
11 # k=2 elegido: inercia cae bruscamente, silueta maxima
12 km = KMeans(n_clusters=2, random_state=42, n_init=20)
13 cluster_temporal = km.fit_predict(X_std)

```

Listing 6: K-Means temporal con validación.

El parámetro `n_init=20` reinicia el algoritmo 20 veces con centroides iniciales distintos y se queda con la mejor solución, evitando mínimos locales.

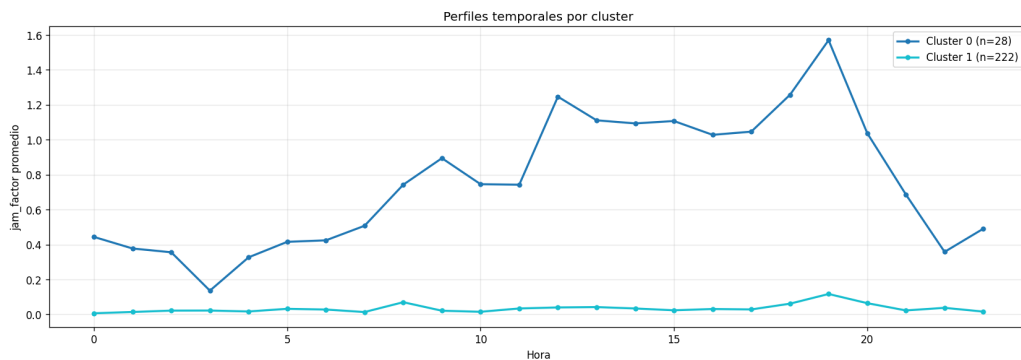


Figura 7: Perfil promedio de `jam_factor` por hora para cada cluster temporal. El cluster 0 (alta congestión) muestra picos pronunciados en hora pico; el cluster 1 (baja congestión) mantiene valores próximos a 1,0 durante todo el día.

4.4.4. Resultados del clustering

Zona	Alta congestión	Baja congestión
La Paz	27 segmentos	223 segmentos
El Alto	10 segmentos	240 segmentos

La congestión severa está concentrada en un subconjunto pequeño de arterias principales: el centro histórico, la Av. Montes y la autopista La Paz–El Alto para La Paz; la Ceja y la Av. 6 de Marzo para El Alto.

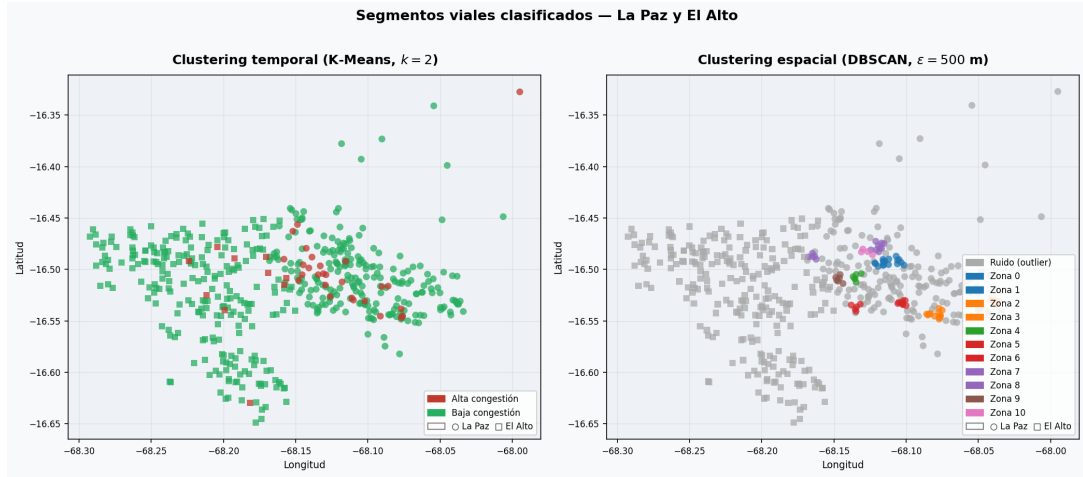


Figura 8: Distribución geográfica de los clusters. **Izquierda:** clustering temporal K-Means (rojo=alta congestión, verde=baja congestión). **Derecha:** clustering espacial DBSCAN (zonas geográficas). Los cuadrados corresponden a El Alto (altiplano, latitudes más altas) y los círculos a La Paz (hoya).

4.5. NMF — Síntesis de días faltantes

4.5.1. El problema

Solo se recolectaron 4 días (lunes, miércoles, viernes, sábado). Martes, jueves y domingo quedaron sin datos. Para completar la semana sin consumir cuota de API, usamos un modelo no supervisado de **factorización matricial**.

4.5.2. Intuición: los arquetipos del tráfico

Imagine que la congestión de una ciudad puede explicarse con 6 “arquetipos” (patrones puros):

1. El **ritmo del centro**: alto mediodía, alto hora pico.
2. El **ritmo de la autopista**: solo en las entradas/salidas del día.
3. El **ritmo residencial**: casi siempre tranquilo, leve pico matutino.
4. El **ritmo comercial**: activo desde las 9 h hasta las 20 h.
5. El **ritmo universitario**: activo en horarios de clases.
6. El **ritmo nocturno**: activo en zonas de vida nocturna.

Cada segmento real es una *mezcla* de estos arquetipos. NMF encuentra los arquetipos (H) y las proporciones de cada segmento (W).

4.5.3. Matemática de NMF

NMF (*Non-negative Matrix Factorization*) descompone la matriz X como:

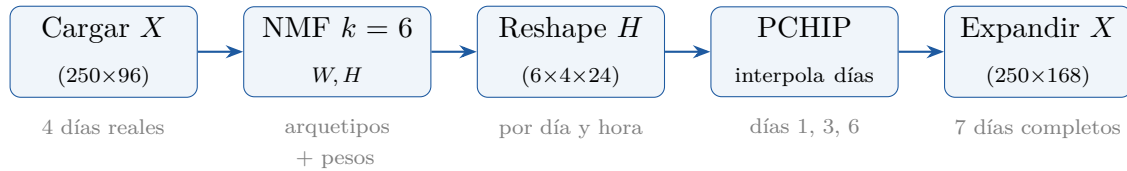
$$X \approx W \cdot H, \quad W \geq 0, H \geq 0 \quad (6)$$

$$\begin{array}{ccc}
 \begin{array}{c} 250 \times 96 \\ \boxed{X} \\ \text{segmentos} \times \text{horas} \end{array} & \approx & \begin{array}{c} 250 \times 6 \\ \boxed{W} \\ \text{pesos} \end{array} \cdot \begin{array}{c} 6 \times 96 \\ \boxed{H} \\ \text{arquetipos} \end{array}
 \end{array}$$

La restricción de **no negatividad** ($W, H \geq 0$) es fundamental: el `jam_factor` siempre es positivo, por lo que tiene sentido que los arquetipos y los pesos sean positivos. PCA, en cambio, genera componentes con valores negativos que no tienen interpretación directa en este contexto.

Característica	NMF	PCA
Valores en componentes	solo positivos	positivos y negativos
Interpretabilidad	alta (partes aditivas)	media
No-negatividad garantizada	sí	no
Unicidad de la solución	no siempre	sí

4.5.4. Pipeline NMF + PCHIP



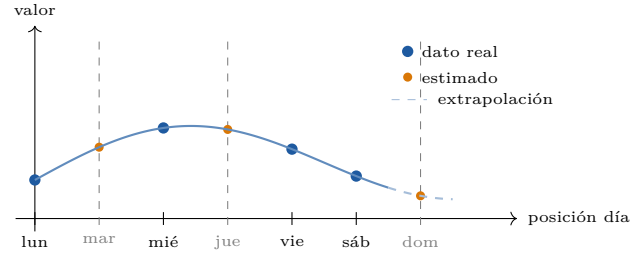
4.6. PCHIP — Interpolación día-de-semana

4.6.1. Intuición: la curva que no oscila

Una vez que NMF nos da los arquetipos para los 4 días conocidos (posiciones 0, 2, 4, 5 en la semana), necesitamos *estimar* los arquetipos para martes (posición 1), jueves (3) y domingo (6).

El problema es conectar puntos con una curva suave. Existen varias opciones:

- **Interpolación lineal:** une los puntos con líneas rectas. Simple, pero poco realista (cambios bruscos).
- **Spline cúbico:** curva muy suave, pero puede oscilar (*ringing*) entre puntos, especialmente al extrapolar fuera del rango conocido.
- **PCHIP** (*Piecewise Cubic Hermite Interpolating Polynomial*): curva cúbica monotone-preserving. Entre dos puntos consecutivos no crea picos o valles artificiales.



El **domingo** (posición 6) queda *fuera* del rango de datos reales (0–5), lo que lo convierte en una **extrapolación**. La extrapolación es inherentemente más incierta que la interpolación (adivinar fuera de los datos conocidos vs. entre ellos), razón por la que el domingo se etiqueta con **confianza baja**.

```

1 from scipy.interpolate import PchipInterpolator
2
3 # Posiciones de los días reales en la semana (0=lunes, 6=domingo)
4 pos_reales = [0, 2, 4, 5] # lunes, mie, vie, sab
5 pos_sint   = [1, 3, 6]    # martes, jueves, domingo
6
7 H_sint = np.zeros((NMF_K, 3, 24))
8 for ki in range(NMF_K):
9     for h in range(24):
10         sp = PchipInterpolator(pos_reales, H[ki, :, h])
11         H_sint[ki, :, h] = sp(pos_sint)
12
13 H_sint = np.clip(H_sint, 0, None) # garantizar no-negatividad

```

Listing 7: Interpolación PCHIP en el eje día-de-semana.

4.6.2. Validación cruzada interna

Antes de sintetizar los días faltantes, evaluamos la calidad del método **ocultando el miércoles** y reconstruyéndolo a partir de {lunes (0), viernes (4), sábado (5)}:

$$\text{RMSE}_{\text{CV}} = \sqrt{\frac{1}{250 \times 24} \sum_{i,h} (X_{i,\text{mié},h} - \hat{X}_{i,\text{mié},h})^2} \quad (7)$$

Zona	RMSE _{CV}	Nivel de confianza
La Paz	0.462	conservador (salto de 4 posiciones)
El Alto	0.435	conservador (salto de 4 posiciones)

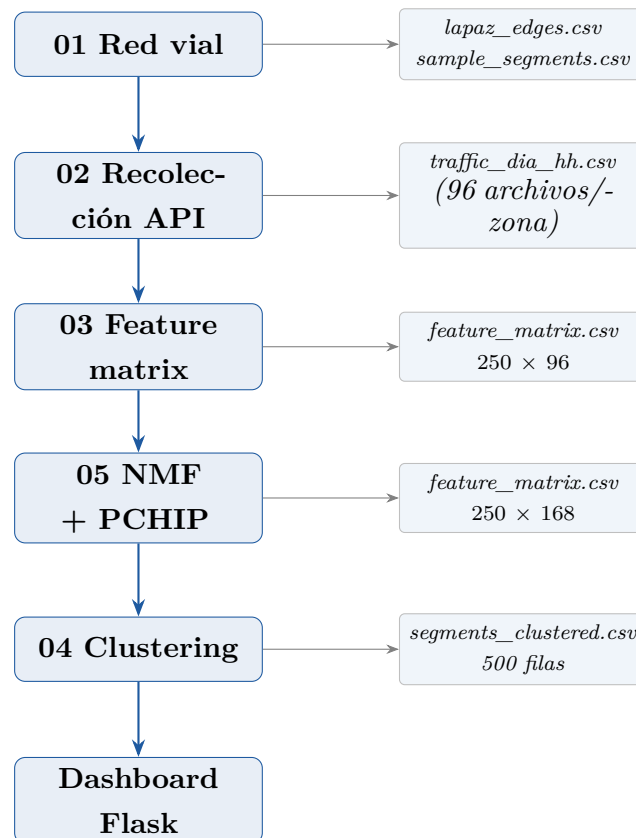
¿Por qué el RMSE es relativamente alto? El CV oculta miércoles (posición 2) y lo predice desde lunes (0), viernes (4) y sábado (5) — un *salto de 4 posiciones*. Martes y jueves tienen días vecinos a distancia 1, por lo que su RMSE real es considerablemente menor. El valor reportado es una **cota superior** conservadora de la calidad.

4.6.3. Nivel de confianza por día

Día	Tipo	Posición	Confianza
Lunes	Real	0	Alta
Martes	Interpolado	1	Media
Miércoles	Real	2	Alta
Jueves	Interpolado	3	Media
Viernes	Real	4	Alta
Sábado	Real	5	Alta
Domingo	Extrapolado	6	Baja

5. Pipeline completo y decisiones de diseño

5.1. Flujo de datos



5.2. Decisiones de diseño no obvias

250 segmentos por zona, no más. La API tiene un costo de ~\$5 por 1 000 consultas. Con 250 segmentos × 24 h × 4 días = 24 000 consultas por zona (≈\$120 total), el costo es manejable y la muestra es representativa.

NMF sobre PCA para síntesis de días. $\text{jam_factor} \geq 1$: nunca puede ser negativo. PCA genera componentes con valores negativos que, al reconstruir, podría producir $\text{jam_factors} < 0$ (sin sentido físico). NMF garantiza que $W, H \geq 0$, por lo que $WH \geq 0$ siempre.

PCHIP sobre spline cúbico. El domingo (posición 6) es extrapolación fuera del rango [0–5]. El spline cúbico estándar puede generar oscilaciones grandes al extrapolar. PCHIP es monotone-preserving: no crea picos artificiales, produciendo una estimación más conservadora y creíble.

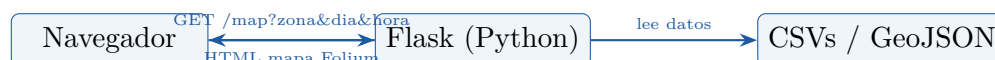
k=6 arquetipos en NMF. Elegido por balance entre interpretabilidad (pocos arquetipos) y fidelidad de reconstrucción (el RMSE de reconstrucción cae rápidamente hasta k=6 y luego muy lentamente). Con k=6 el RMSE de reconstrucción baja de 0.03 (vs. $\text{jam_mean} \approx 0.18$).

DBSCAN con eps=500 m. 500 m es la distancia a pie en ~ 6 minutos: calles que están en el mismo “barrio” a escala peatonal. A esta escala, los segmentos del centro histórico forman un cluster coherente separado de los barrios periféricos.

6. Dashboard interactivo

6.1. Arquitectura

El dashboard está construido con **Flask** (servidor web Python) y **Folium** (mapas Leaflet.js interactivos en el navegador).



Cuando el usuario mueve el slider de hora, el JS del navegador hace un `fetch()` a `/map`, Flask construye un mapa Folium en memoria y devuelve el HTML como respuesta. El mapa se embebe en un `<iframe>` y la comunicación entre iframe y página usa `postMessage`.

6.2. Extensión de calles por nombre

Un problema visual: solo 250 de $\sim 37\,100$ segmentos están en la muestra, lo que hace que las avenidas aparezcan como puntos aislados en el mapa. Solución: se carga un GeoJSON con *todos* los segmentos de la red, y a cada uno que comparte nombre con un segmento muestreado se le asigna el color de ese segmento (con opacidad reducida). Esto extiende visualmente las avenidas sin duplicar datos de tráfico.

6.3. Modos de visualización

Por hora: muestra el `jam_factor` para el día y hora seleccionados. Verde → amarillo → naranja → rojo según el nivel de congestión.

Promedio histórico: muestra el `jam_mean` de cada segmento calculado como la media de las 168 columnas (7 días × 24 h). Identifica arterias crónicamente congestionadas.

6.4. Transparencia: datos estimados

Los días martes, jueves y domingo aparecen como “**Martes (est.)**” en el selector y un aviso amarillo se muestra al seleccionarlos. Esto garantiza que cualquier usuario sepa que esos datos son *estimaciones estadísticas*, no mediciones reales.

7. Resultados

7.1. Patrones encontrados

- El **cluster de alta congestión** en La Paz agrupa calles del centro histórico, la Av. Montes y los accesos a la autopista La Paz–El Alto; en El Alto, la Ceja y la Av. 6 de Marzo.
- El **cluster de baja congestión** abarca barrios residenciales periféricos donde el tráfico no supera `jam_factor` $\approx 1,3$ incluso en hora pico.
- La proyección UMAP (Figura 5) revela dos grupos claramente separados, confirmando que la diferencia de patrones es genuina y no un artefacto del algoritmo.
- Los días sintéticos (martes, jueves) producen perfiles coherentes con los días reales adyacentes.

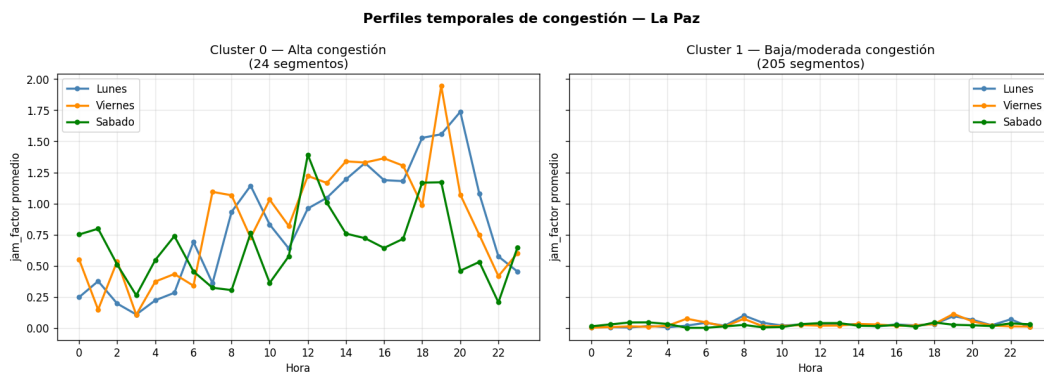


Figura 9: Curvas de congestión por hora del día para los dos clusters, en los distintos días recolectados. El cluster de alta congestión muestra picos pronunciados a las 8 h y 17 h; el de baja congestión se mantiene plano.

7.2. Limitaciones

1. **Muestra.** 250 de $\sim 37\,100$ segmentos; algunas calles importantes pueden no estar representadas.
2. **Datos históricos de Google.** Estimaciones basadas en datos anónimos; pueden diferir en eventos especiales (feriados, bloqueos).
3. **Días estimados.** Martes, jueves y especialmente domingo son síntesis estadísticas; no deben usarse para decisiones críticas.
4. **Estacionalidad.** Los datos corresponden a un período acotado de 2024–2025; no capturan variaciones estacionales.

8. Conclusiones

Este proyecto demuestra que el aprendizaje no supervisado permite extraer conocimiento valioso sobre el comportamiento del tráfico urbano *sin necesidad de datos etiquetados ni expertos previos*.

La combinación de técnicas resultó más rica que cualquier método aislado:

- **PCA** redujo el ruido y comprimió 96 dimensiones a 13, preservando el 86–96% de la información.
- **UMAP** hizo visible la separación entre grupos en 2D.
- **DBSCAN** identificó vecindades geográficas sin asumir formas de cluster ni requerir k .
- **K-Means** segmentó los patrones temporales en grupos semánticamente interpretables.
- **NMF + PCHIP** completó la semana preservando la no-negatividad del `jam_factor` y cuantificando la incertidumbre.

El resultado práctico —el dashboard interactivo— pone esta información al alcance de cualquier ciudadano o gestor de movilidad, con etiquetas claras sobre la fiabilidad de cada dato.

Trabajo futuro. Con datos históricos de más días y semanas sería posible aplicar modelos de series temporales (LSTM, Prophet) para predicción en tiempo real, e incorporar datos de precipitación o eventos para modelos de impacto vial.

A. Dependencias del proyecto

```
1 flask>=3.0
2 pandas>=2.0
3 numpy>=1.26
4 scikit-learn>=1.4
5 scipy>=1.12
6 umap-learn>=0.5
7 folium>=0.16
8 osmnx>=1.9
9 plotly>=5.20
10 matplotlib>=3.8
```

Listing 8: requirements.txt resumido.

B. Estructura de archivos

```
1 lapaz_traffic/
2   data/
3     raw/
4       sample_segments.csv           # 250 segmentos La Paz
5       sample_segments_el_alto.csv   # 250 segmentos El Alto
6       traffic_lunes_00.csv ...       # datos recolectados
7       el_alto/traffic_*.csv
8     processed/
9       feature_matrix.csv             # 250 x 168
10      feature_matrix_el_alto.csv      # 250 x 168
11      segments_clustered.csv          # 500 filas con clusters
12      synthetic_days_metadata.csv      # confianza por dia
13 notebooks/
14   01_red_vial.ipynb
15   02_recoleccion.ipynb
16   03_preprocesamiento.ipynb
17   04_reduccion_dimensionalidad.ipynb
18   05_clustering.ipynb
19   06_visualizacion.ipynb
20 scripts/
21   03_rebuild_features.py
22   04_rebuild_clustering.py
23   05_predict_missing_days.py
24 templates/
25   index.html
26   map.html
27 dashboard.py
28 informe/
29   informe.tex
30   informe.pdf
31   gen_map.py
```

```
32  img/  
33    red_vial.png  
34    perfiles_raw.png  
35    preprocesamiento.png  
36    pca_varianza.png  
37    umap_proyeccion.png  
38    mapa_clusters.png  
39    kmeans_codo.png  
40    perfiles_clusters.png  
41    perfiles_24h.png
```

Listing 9: Árbol de directorios del proyecto.